

LJSF 3 - ATLAS Installation System

Complete operations and administration manual



Item	Value
Name	LJSF 3 - ATLAS Installation System
Version	3.0.0
Package revision	r32
Creator / maintainer	Alessandro De Salvo
License	EUPL-1.2
Web root	/atlas_install/
REST API	/atlas_install/api/v1/
Reference platform	Rocky Linux 10 / PHP 8.3+ / Kubernetes / MariaDB-Galera

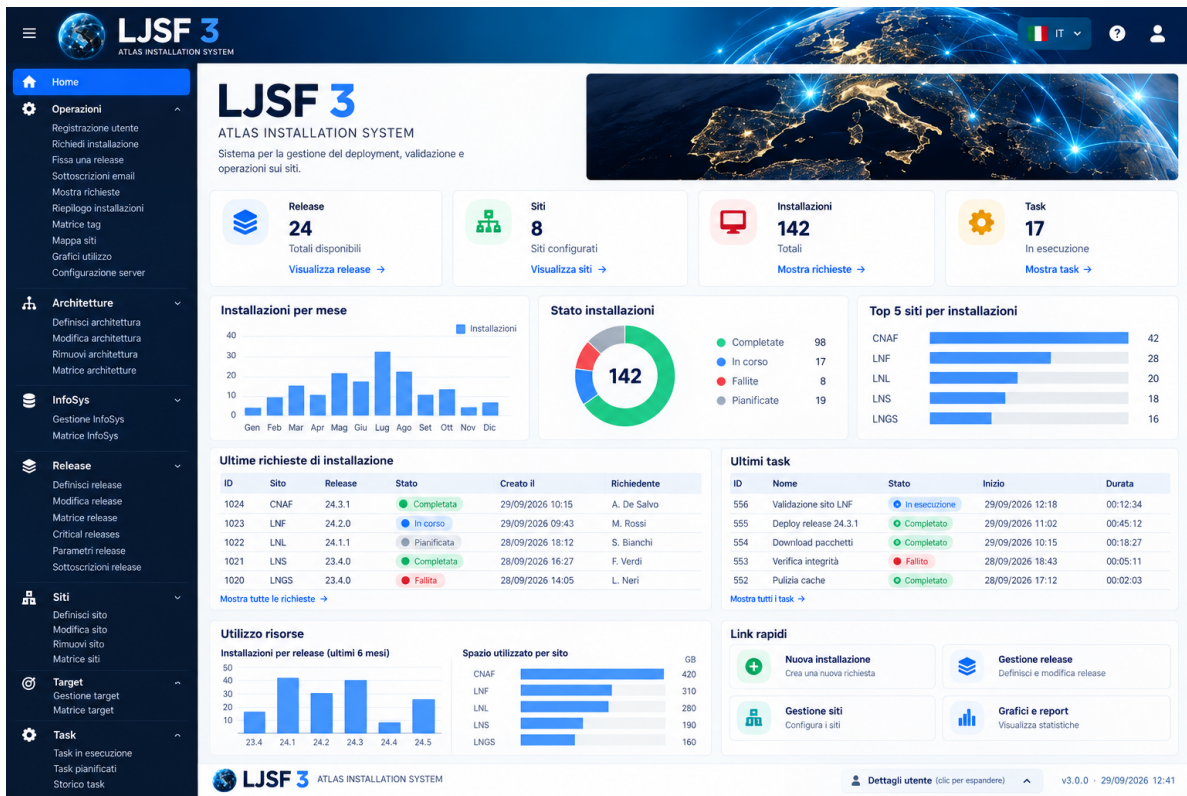
This manual describes the current application behavior and is not a changelog. Example hostnames, users and application data are illustrative.

Structured contents

1. Purpose and operating model
2. System architecture
3. Kubernetes topology and networking
4. Container startup and readiness
5. Configuration and Secrets
6. Database, schema and migrations
7. Authentication, roles and authorization
8. TOTP and QR codes
9. Web navigation and responsive UI
10. Releases and architectures
11. Sites, InfoSys, targets and tasks
12. Installation requests
13. Installation status and list.php
14. Charts, summaries and CronJobs
15. Logging and observability
16. REST API - model and usage
17. Operational security
18. Backup, restore and compatibility
19. Troubleshooting
20. Development and GitHub
21. Appendix A. Configuration variables
22. Appendix B. REST endpoints
23. Appendix C. Glossary

1. Purpose and operating model

LJSF 3 manages the ATLAS software-deployment lifecycle: release and architecture definitions, sites and resources, installation requests, operational tasks, installation status, subscriptions, reports and administrative functions. The application preserves legacy interfaces and adds a versioned REST API for new clients.

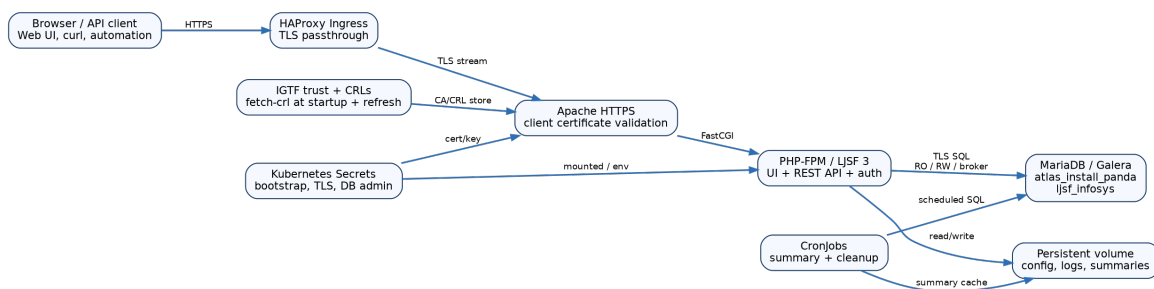


UI overview

Illustrative LJSF 3 UI screenshot with demonstration data.

2. System architecture

Browser/API traffic enters through HAProxy Ingress in TLS passthrough mode. Apache terminates TLS, validates client certificates according to path policy, and forwards PHP to PHP-FPM. The app uses separate MariaDB connections for read, write and broker access; the migrator can use a separate administrative credential. The persistent volume stores managed configuration, logs and summaries.

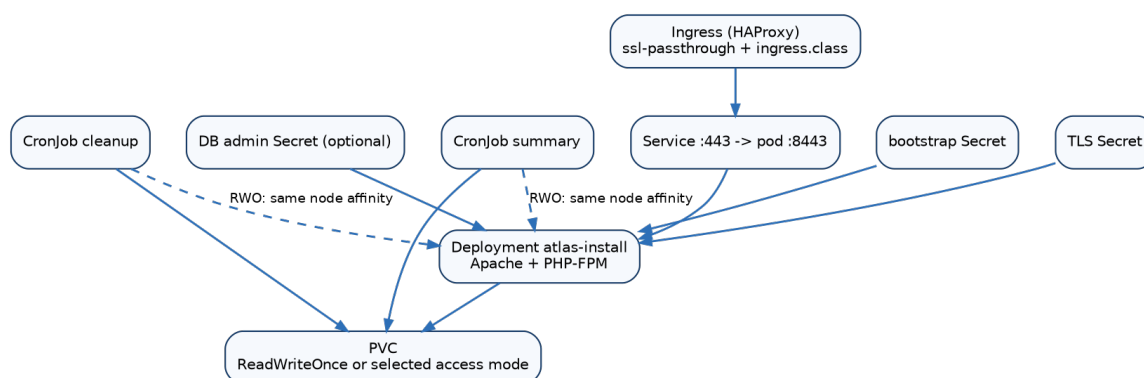


System architecture

3. Kubernetes topology and networking

The Deployment exposes HTTPS on its configured container port and the Service publishes 443. HAProxy manifests include both `spec.ingressClassName: haproxy` and

`kubernetes.io/ingress.class: haproxy`, plus SSL passthrough. With a `ReadWriteOnce` PVC, CronJobs mounting the same volume are co-located on the server node.



Kubernetes topology

3.1 Main Kubernetes objects

- **Deployment** runs Apache and PHP-FPM and mounts TLS, bootstrap configuration and the PVC.
- **Service** publishes the pod HTTPS endpoint inside the cluster.
- **HAProxy Ingress** performs TLS passthrough so Apache receives the original client certificate.
- **PVC** stores managed configuration, logs, caches and summaries.
- **Summary/Cleanup CronJobs** calculate local reporting data and enforce retention.
- **Bootstrap/TLS/DB-admin Secrets** separate runtime configuration, server credentials and optional schema-migration privilege.

3.2 Post-rollout checks

```
kubectl -n atlas-install rollout status deploy/atlas-install
kubectl -n atlas-install get pods -o wide
kubectl -n atlas-install get ingress,svc,pvc,cronjob
kubectl -n atlas-install get events --sort-by=.lastTimestamp | tail -50
```

4. Container startup and readiness

1. configuration sync from bootstrap Secret
2. local-auth encryption key
3. HTTPS certificate validation
4. DB/auth-schema check and migration
5. IGTF trust restore and initial fetch-crl
6. Apache render/validation
7. PHP-FPM startup
8. Apache startup `healthz.php` shows that the Web process is alive. `readyz.php` also validates DB readiness; do not confuse liveness with readiness.

4.1 Startup diagnostics

Startup logs expose configuration sync, DB migration, IGTF/fetch-crl, PHP-FPM and Apache stages. If the container never starts and logs are empty, inspect `kubectl describe pod` and cluster events for mount, Multi-Attach, Secret or scheduling failures.

```
kubectl -n atlas-install exec deploy/atlas-install -- curl -fsS http://127.0.0.1/atlas_install/healthz.php
kubectl -n atlas-install exec deploy/atlas-install -- curl -fsS http://127.0.0.1/atlas_install/readyz.php
```

5. Configuration and Secrets

The bootstrap Secret is the authoritative source for `atlas-install.env`. The entrypoint atomically synchronizes the persistent file when the Secret changes. DB passwords and TLS keys are not stored in the wizard local state. The DB-admin migration credential is optional and separate from normal `dbwriter` access.

5.1 Kubernetes wizard

The wizard is idempotent: it reuses non-secret choices as defaults, preserves existing passwords unless changed, and restarts the Deployment only when a change requires reload. It supports optional DB TLS, PVC size/access mode, ingress class, CPU/memory resources and separated DB credentials.

5.2 Database TLS

`ATLAS_DB_SSL=1` enables TLS for application DB connections. `ATLAS_DB_SSL_VERIFY=1` enables server certificate verification and `ATLAS_DB_SSL_CA` identifies the CA. Compare a pod-local `mariadb --ssl` test with PHP behavior when troubleshooting. A Kubernetes Secret is base64-encoded, not automatically encrypted at rest; actual at-rest encryption requires API-server/etcd encryption configuration.

6. Database, schema and migrations

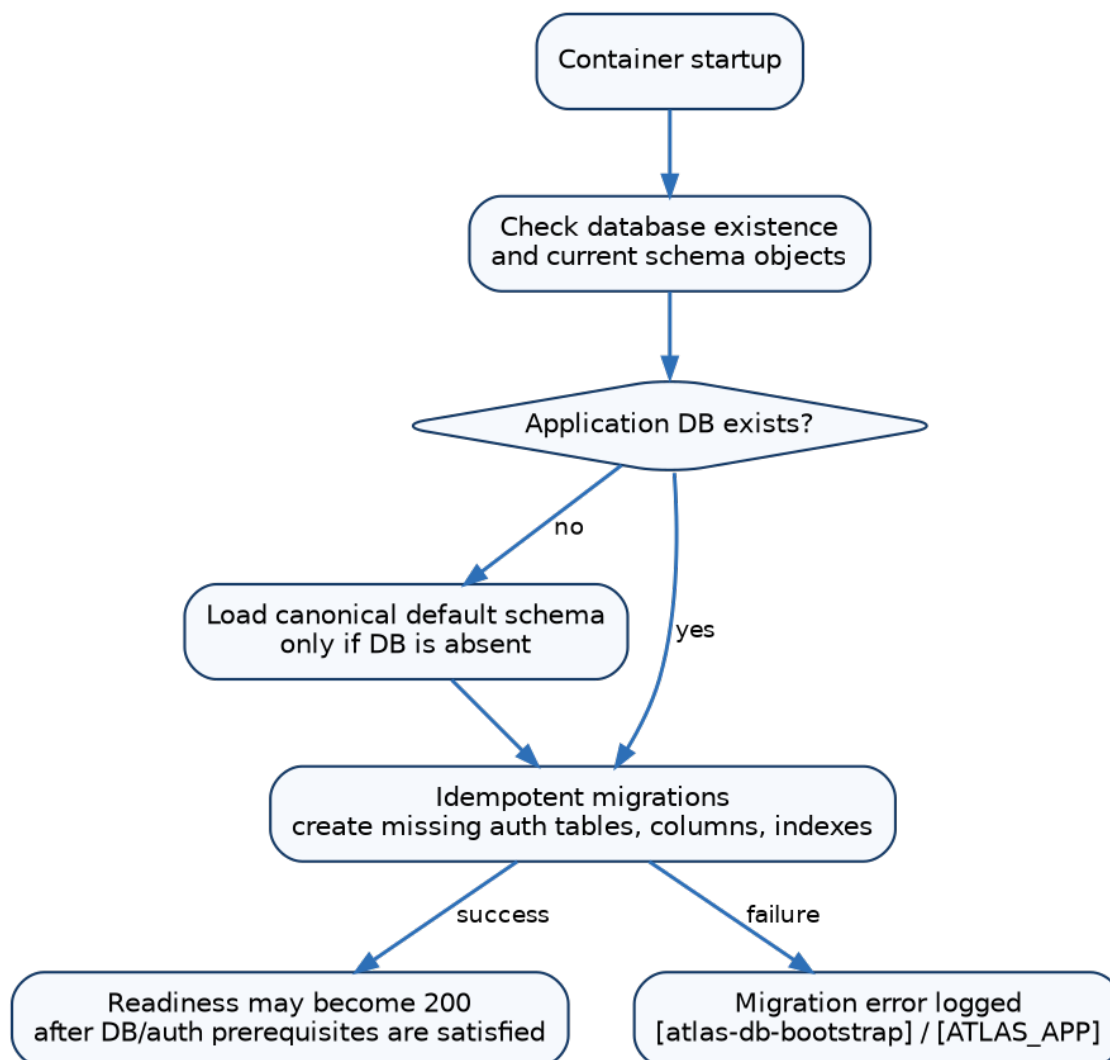
Startup migrations are idempotent: after restoring an older dump, the application adds missing objects required by the current version (auth tables, columns, indexes and migration markers) without recreating an existing database. The canonical full schema is loaded only when the application database is absent.

6.1 Database credential separation

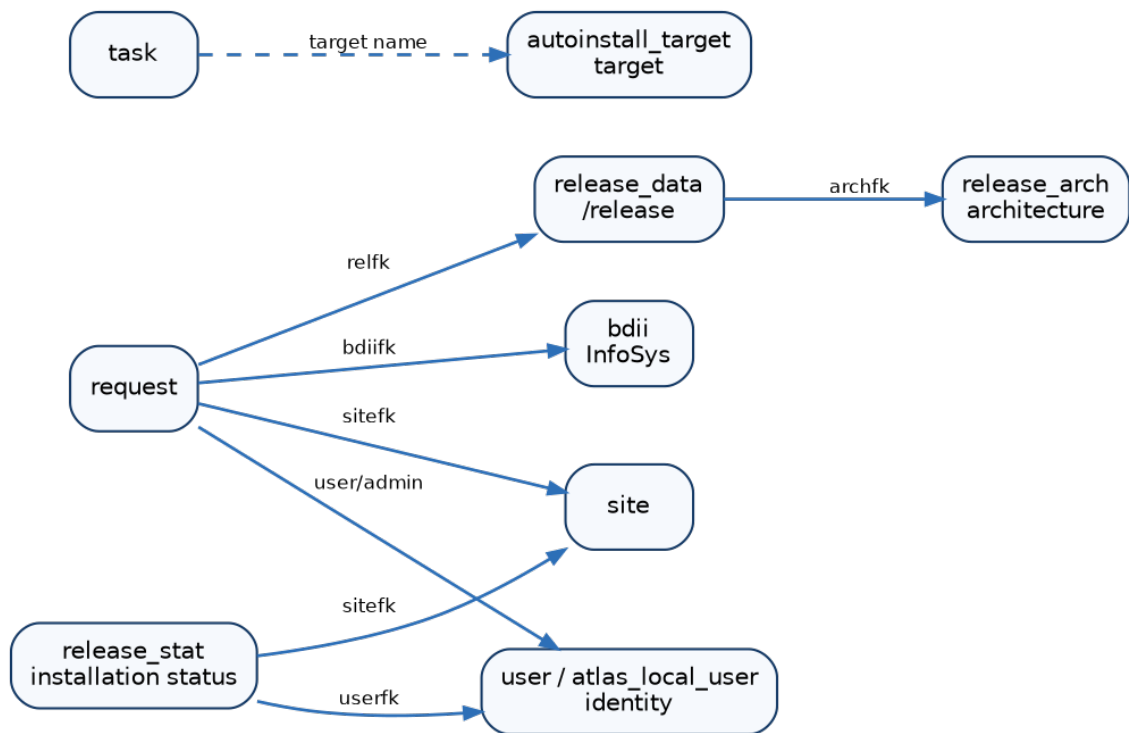
- **RO** for reads/readiness.
- **RW** for normal application writes.
- **Broker** for dedicated legacy execution paths.
- **Bootstrap/admin** for DDL and schema migration only.

6.2 Restoring an older dump

Restore data, preserve/recreate Secrets, start with `ATLAS_DB_AUTO_INIT=1`, inspect migration logs, verify auth tables/columns/indexes, wait for readiness, then test local login and protected operations before reopening production writes.



Schema migration flow



Core data model

7. Authentication, roles and authorization

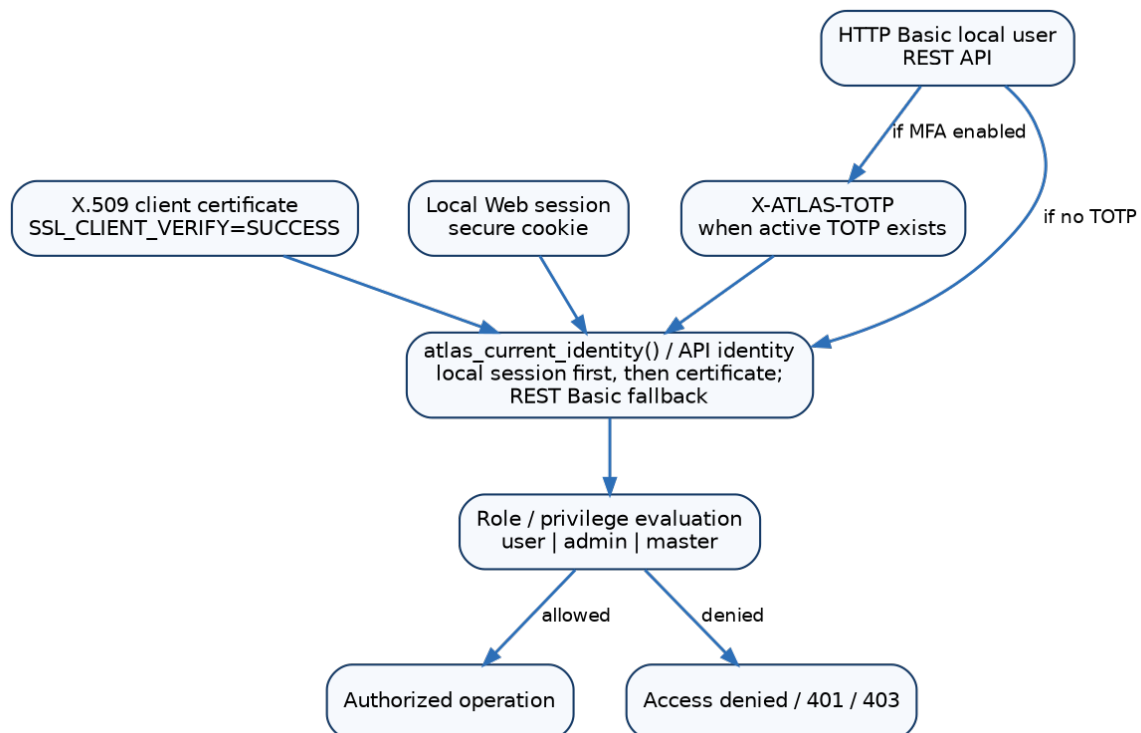
X.509 client certificates and local accounts are supported. Local roles are `user`, `admin`, `master`. Protected pages keep header/menu on denial and direct the user to local login or an appropriate certificate. For legacy compatibility, an authenticated local account is synchronized to a `LOCAL:<username>` identity in the historical `user` table.

7.1 Conceptual access matrix

Identity	Public reads	Protected functions	User/config administration
Anonymous	Where exposed	No	No
Valid but unmapped certificate	Public only	No until mapped	No
Local user	Yes	Role-permitted operations	No
<code>admin</code>	Yes	Normal administrative operations	Function-dependent
<code>master</code>	Yes	Yes	Yes

7.2 Client certificates

TLS passthrough lets Apache validate the client certificate directly. The normalized DN is mapped through the historical `user / role` tables; certificate validity alone does not grant application privileges.



Authentication flow

8. TOTP and QR codes

Local accounts can enroll one or more TOTP authenticators. The secret is stored encrypted; enrollment shows both the Base32 string and a locally generated QR code using an `otpauth://` URI. The QR is not sent to external services. Verification accepts the current time slice plus adjacent slices to tolerate small clock skew.

8.1 TOTP enrollment

Generate the secret, display Base32 and QR, scan it with an authenticator, verify a code, then enable the credential. Keep cluster/device time synchronized with NTP because clock drift is a common source of rejected codes.

9. Web navigation and responsive UI

Language follows the browser (`it*` -> Italian, otherwise English) and can be overridden using the selector. On mobile the menu is a vertical drawer. Wide tables become collapsible records: collapsed rows show priority fields, expanded rows show details vertically. Browser views default to 200 rows per page; this does not constrain API or machine-oriented queries.

10. Releases and architectures

Releases are defined in `release_data`; architectures in `release_arch`. Protected functions support define/update/delete subject to DB dependencies. Core fields include name/tag/package, architecture, obsolete/autoinstall flags, software paths, criticality and validity windows.

11. Sites, InfoSys, targets and tasks

Sites contain ATLAS/grid identity, OS/architecture, filesystem, capacity and status. InfoSys (`bdi`) stores endpoints and preference/enable flags. Targets define auto-install destinations; tasks define reusable operations. Legacy definition pages remain for compatibility and are authorization-protected.

12. Installation requests

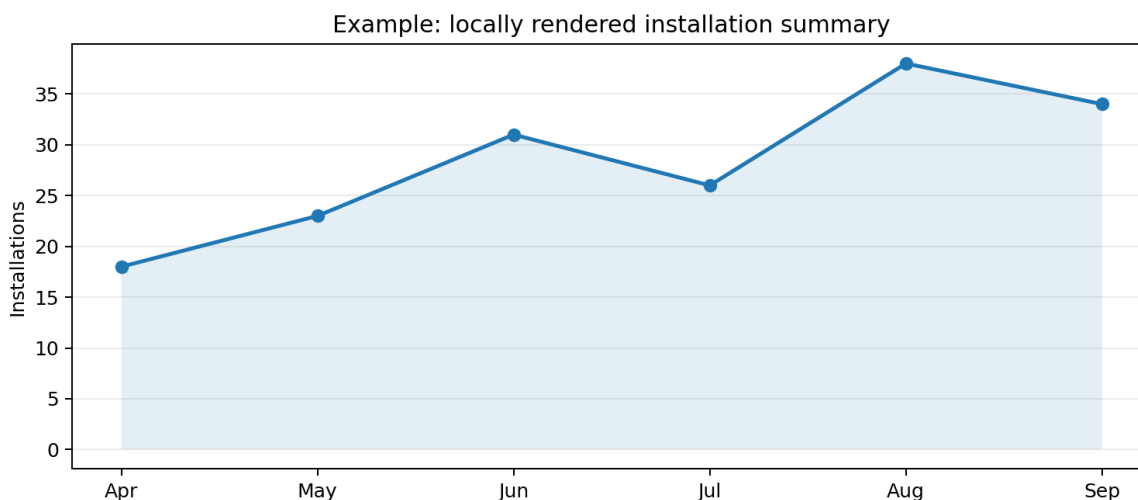
`protected/req.php` displays and manages requests. The query is paginated and optimized with explicit joins and dedicated indexes. Slow query events are logged as `slow_database_query`. Requests reference site, release, type, requester, assigned administrator and status.

13. Installation status and `list.php`

`list.php` provides search, filters and sorting in a collapsed section. Records are collapsible on desktop as well: collapsed rows show at least Num, Release number, Site name and Release arch; details expand without forcing horizontal scrolling. Historical status colors are preserved. Footer and identity details are separate from the data frame.

14. Charts, summaries and CronJobs

CronJobs do not generate images through external services. They produce summaries/cache; the Web UI renders charts locally on demand as SVG/HTML. This removes external rendering dependencies and keeps rendering deterministic inside the cluster.



Local chart example

15. Logging and observability

The app writes structured `[ATLAS_APP]` events with `request_id`, `URI` and event name, excluding passwords/secrets. Persistent `/var/lib/atlas-install/log/php-application.log` is tailed by the entrypoint to `stderr` so `kubectl logs` also receives PHP-FPM errors. Key events include `php_error`, `uncaught_exception`, `http_5xx_completed`, `same_origin_rejected`, `slow_database_query`, `auth` events and diagnostic checkpoints for critical pages.

15.1 Correlation and commands

A valid incoming `X-Request-ID` is reused; otherwise the application creates one. Use the request id shown in an error/response to locate the matching log event.

```
kubectl logs -n atlas-install deploy/atlas-install -f
kubectl logs -n atlas-install deploy/atlas-install --since=30m | grep '\[ATLAS_APP\]'
kubectl exec -n atlas-install deploy/atlas-install -- tail -200 /var/lib/atlas-install/log/
php-application.log
```

```
kubectl logs -n atlas-install deploy/atlas-install --since=10m -f
kubectl exec -n atlas-install deploy/atlas-install -- tail -100 /var/lib/atlas-install/log/
php-application.log
```

16. REST API - model and usage

REST API v1 is parallel to legacy endpoints. It uses resource URIs, JSON, standard HTTP methods, status codes and request IDs. See `REST-API.it.md/REST-API.en.md` and this manual API appendix for the full reference.

LJSF 3 REST API - v1

JSON, resource-oriented, versioned API

Request

```
GET /atlas_install/api/v1/releases?limit=2&sort=name&order=DESC
Host: atlas-install.example.org
Accept: application/json
X-Request-ID: demo-7c3f2a
```

200 OK

```
{
  "data": [
    {"ref": 1042, "name": "24.3.1", "tag": "AtlasOffline_24_3_1"},
    {"ref": 1037, "name": "24.2.0", "tag": "AtlasOffline_24_2_0"}
  ],
  "meta": {"total": 24, "limit": 2, "offset": 0, "count": 2}
}
```

Authenticated request

```
curl -u operator:**** \
-H 'X-ATLAS-TOTP: 123456' \
https://atlas-install.example.org/atlas_install/api/v1/re
```

Error envelope

```
403 Forbidden
{
  "error": {
    "status": 403,
    "code": "forbidden",
    "message": "The master role is required for this resource.",
    "request_id": "demo-7c3f2a",
    "details": {}
  }
}
```

Illustrative values only. Local password hashes are never returned.

REST API

17. Operational security

- Require TLS for public traffic and credential-bearing API calls.
- Configure database TLS and CA verification.
- Use least privilege for RO/RW/broker/migrator accounts.
- Never log passwords, tokens, TOTP codes or private keys.
- Keep IGTF/CRLs current and verify fetch-crl in startup logs.
- Enable Kubernetes Secret encryption at rest.
- Grant master only to appropriate operators/administrators.
- Monitor 401/403/5xx and slow queries.

18. Backup, restore and compatibility

A restore can move the DB back to an older schema: keep the startup migrator enabled so missing tables/indexes/columns are recreated. Migrations must not delete data. Before major restores preserve Secrets, TLS, PVC and configuration; after restore verify readiness, migrator logs, auth tables, indexes and local login.

19. Troubleshooting

Symptom	Check
Pod not Ready	Compare <code>/readyz.php</code> , startup logs, DB TLS, auth schema and bootstrap Secret.
Web 500	Use <code>X-Request-ID</code> and search <code>[ATLAS_APP]</code> ; inspect the persistent log file too.
Local login fails	Check enabled account, must-change-password, TOTP and auth tables; distinguish from same-origin/CSRF.
Certificate denied	Check TLS passthrough, <code>SSL_CLIENT_VERIFY</code> , normalized DN and user mapping.
req.php is slow	Search <code>slow_database_query</code> , inspect indexes/ cardinality; do not raise timeout as the first fix.
CronJob Pending with RWO	Verify podAffinity to server node and Multi-Attach events.
Wrong language	Check <code>lang.php</code> preference and <code>Accept-Language</code> .

20. Development and GitHub

The canonical repository contains Dockerfile/Containerfile, Kubernetes manifests, wizard, static tests and documentation. Every revision should run PHP/shell lint, YAML parsing, Secret-hygiene checks, UI/API regression tests and multiarch CI build. REST v1 contract changes must be backward-compatible or use a new API version.

ewpage ## Appendix A. Main configuration variables

The following variables are the primary operating contract between the wizard, Kubernetes Secrets, the entrypoint and the application. Passwords are listed only by variable name: their values must not appear in deployment documentation, logs or wizard state files.

Variable	Purpose
<code>ATLAS_PUBLIC_HOSTNAME</code>	Public DNS hostname used by Apache/TLS and same-origin checks.
<code>ATLAS_HTTPS_PORT</code>	HTTPS port inside the container (normally 8443).
<code>ATLAS_ENV_FILE</code>	Managed runtime environment file path.
<code>ATLAS_BOOTSTRAP_ENV</code>	Mounted bootstrap Secret file used to refresh the managed env file.
<code>ATLAS_BOOTSTRAP_SYNC_SECONDS</code>	Interval used to detect bootstrap Secret changes.
<code>ATLAS_DB_NAME</code>	Application database name, normally <code>atlas_install_panda</code> .
<code>ATLAS_DB_RW_HOST / USER / PASSWORD</code>	Read-write database endpoint and credentials.
<code>ATLAS_DB_RO_HOST / USER / PASSWORD</code>	Read-only database endpoint and credentials.

Variable	Purpose
ATLAS_DB_BROKER_HOST / USER / PASSWORD	Broker/legacy execution database endpoint and credentials.
ATLAS_DB_BOOTSTRAP_USER / PASSWORD	Optional privileged credential used by startup schema migration.
ATLAS_DB_AUTO_INIT	Enable idempotent application/auth schema initialization at startup.
ATLAS_DB_SSL	Enable TLS for MariaDB connections.
ATLAS_DB_SSL_VERIFY	Enable server-certificate verification for DB TLS.
ATLAS_DB_SSL_CA	CA bundle/path used to verify the MariaDB server certificate.
ATLAS_DB_SLOW_QUERY_MS	Threshold for slow_database_query logging.
ATLAS_REQUEST_TIMEOUT_SECONDS	Application-side timeout margin for long protected request pages.
ATLAS_TLS_CERT_FILE	Server certificate/full-chain file mounted in the container.
ATLAS_TLS_KEY_FILE	Server private key file mounted in the container.
ATLAS_IGTF_DIR	Active IGTF trust-anchor/CRL directory.
ATLAS_IGTF_CACHE_DIR	Cache used by IGTF refresh logic.
ATLAS_IGTF_REFRESH_SECONDS	Periodic IGTF/CRL refresh interval.
ATLAS_CRL_MAX_AGE_SECONDS	Maximum accepted CRL age.
ATLAS_CRL_FETCH_TIMEOUT_SECONDS	Timeout used for CRL retrieval.
ATLAS_LOCAL_AUTH_KEY_FILE	Local-auth encryption key file for encrypted TOTP secrets.
ATLAS_APP_ERROR_LOG	Persistent PHP/application log path tailed into container stderr.
ATLAS_UPLOAD_PATH	Persistent upload/data path.
ATLAS_ARCHIVE_PATH	Archive/log backup path.
ATLAS_CACHE_PATH	Application cache path.
ATLAS_KML_CACHE	KML cache path.
ATLAS_VO	VO label, normally ATLAS.
ATLAS_EMAIL	Application sender/contact email setting.
ATLAS_CONTACTS	Installation-team contact string.
ATLAS_DEFAULT_INFOSYS	Default InfoSys/BDII endpoint.
ATLAS_ACTIVITY_PERIOD	Default activity period used by legacy views.
ATLAS_DEBUG	Enable verbose diagnostic behavior; not for normal production use.

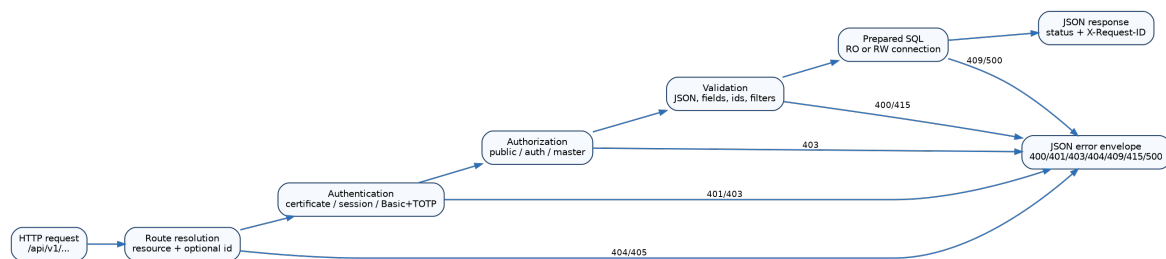
Legacy software-agent variables can also exist for compatibility; do not confuse them with Web-server runtime configuration. Before changing a production value, verify whether it is wizard-managed and therefore refreshed from the bootstrap Secret.

Appendix B. REST API - complete reference

Normative reference for REST API v1 included in package r32. Legacy endpoints remain available but are outside the REST contract.

1. Service identity

- **Product:** LJSF 3 - ATLAS Installation System
- **Application version:** 3.0.0
- **Package revision:** r32
- **Base path:** /atlas_install/api/v1
- **Media type:** application/json; charset=UTF-8
- **Response cache:** Cache-Control: no-store
- **API version header:** X-API-Version: v1
- **Request correlation:** X-Request-ID

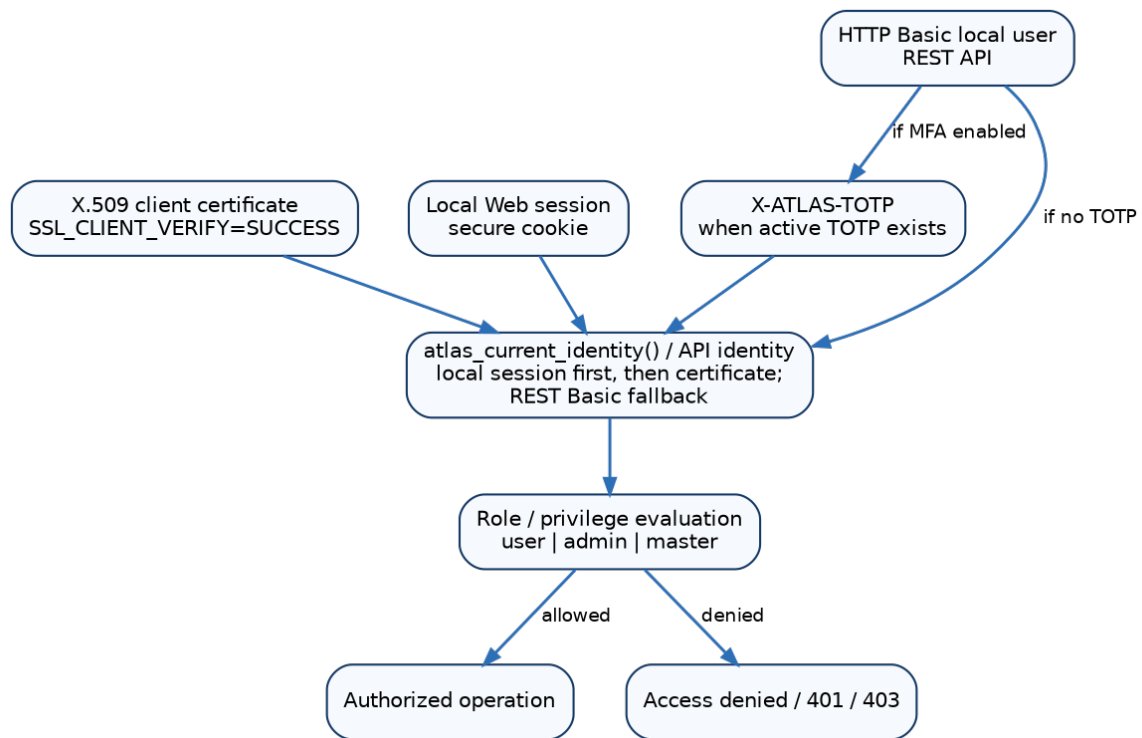


REST request lifecycle

Figure: REST request lifecycle.

2. Authentication model

The API reuses the application identity model. Identity resolution is: an existing local Web session; a valid X.509 client certificate; then, as a REST-specific fallback, HTTP Basic using a local account. If the local account has at least one active TOTP credential, Basic authentication also requires X-ATLAS-TOTP.



Authentication model

Mechanism	Transport	Requirements	Notes
X.509 certificate	TLS client certificate	TLS reaches Apache; SSL_CLIENT_VERIFY=SUCCESS; DN mapped in user.	Suitable for personal/robot certificates.
Local session	Secure Web cookie	Unexpired local session and enabled account.	Useful for browser-originated API calls.
HTTP Basic	Authorization: Basic ...	Enabled local account, valid password, must_change_password=0.	Use only over HTTPS.
Basic + TOTP	Basic + X-ATLAS-TOTP: 123456	Required when at least one active TOTP exists.	Six digits; current TOTP slice +/- 1 is accepted.

2.1 Authentication examples

```
# Basic without TOTP
curl --fail-with-body -u operator:'PASSWORD' \
https://atlas-install.example.org/atlas_install/api/v1/me

# Basic with TOTP
curl --fail-with-body -u operator:'PASSWORD' \
-H 'X-ATLAS-TOTP: 123456' \
https://atlas-install.example.org/atlas_install/api/v1/me

# X.509
curl --fail-with-body --cert client.pem --key client.key \
https://atlas-install.example.org/atlas_install/api/v1/me
```

2.2 Authentication failures

- 401 authentication_required: no valid identity. Response includes WWW-Authenticate: Basic realm="LJSF 3 REST API".
- 401 totp_required: Basic credentials are valid but TOTP is missing/invalid.
- 403 password_change_required: the local account must change its password through the Web UI before API use.
- 403 forbidden: valid authentication but insufficient role.

3. HTTP and JSON conventions

Item	Behavior
GET collection	200 with {data:[...], meta:{resource,total,limit,offset,count}}
GET item	200 with {data:{...}}; 404 if not found
POST collection	Requires JSON body; 201 Created; Location header; response contains created id and location
PATCH item	Partial update; 200 with the updated resource
PUT item	Currently has the same supplied-field partial-update semantics as PATCH; it is not a full replacement
DELETE item	204 No Content; 404 if missing; 409 on database conflicts
OPTIONS	Returns supported method names; router advertises GET, HEAD, POST, PUT, PATCH, DELETE, OPTIONS
HEAD	Accepted on read routes; web server may suppress the body as required by HTTP
Content-Type	POST/PUT/PATCH with a non-empty body require application/json; otherwise 415

Item	Behavior
Unknown JSON field	400 unknown_field; immutable keys are also rejected on update
Arrays/objects in fields	Rejected with 400 invalid_field; resource fields are scalar
Caching	All API responses send Cache-Control: no-store

3.1 Error envelope

```
{
  "error": {
    "status": 400,
    "code": "invalid_request",
    "message": "...",
    "request_id": "7c3f2a9b...",
    "details": {}
  }
}
```

3.2 Status codes

HTTP	Name	Meaning
200	OK	Successful read or update.
201	Created	Successful creation; use Location.
204	No Content	Successful deletion.
400	Bad Request	Invalid id/JSON/field/request.
401	Unauthorized	Missing authentication or TOTP required.
403	Forbidden	Insufficient role or password change required.
404	Not Found	Unknown route/resource/record.
405	Method Not Allowed	Method unsupported by route.
409	Conflict	Database constraint/conflict during write/delete.
415	Unsupported Media Type	Non-JSON request body.
500	Internal Server Error	Internal error; correlate request_id with logs.

4. Collections, filtering, search, pagination and sorting

Every exposed field can be supplied as a query parameter for exact matching (`field=value`). `q` performs `LIKE %q%` only on the resource search fields. `limit` defaults to 200 and is capped at 1000; `offset` cannot be negative. `sort` must be an exposed field or the primary key is used; `order` accepts `ASC` or `DESC` , otherwise `ASC` .

```
curl 'https://atlas-install.example.org/atlas_install/api/v1/releases?
      limit=100&offset=0&sort=name&order=DESC&q=24.3'
```

5. Resource matrix

Resource	DB table	Key	GET	Write	fields
releases	release_data	ref	public	authenticated user	name, tag, package, comments
sites	site	ref	public	authenticated user	cs, cename, name, atlas_name, alias, arch
requests	request	id	authenticated user	authenticated user	id, user_comments, admin_comments
tasks	task	ref	public	authenticated user	name, description, target
architectures	release_arch	ref	public	authenticated user	platform_type, os_type, gcc_ver, mode, description
targets	autoinstall_target	ref	public	authenticated user	name, description
infosys	bdii	ref	public	authenticated user	hostname, ns, lb, ld, wmpoxy, myproxy
release-subscriptions	release_subscription	ref	authenticated user	authenticated user	sitename, pattern, comment
release-status	release_stat	ref	public	authenticated user	name, tag, status, comments

Resource	DB table	Key	GET	Write	fields
grids	grid	ref	public	authenticated user	name, description
facilities	facility	ref	public	authenticated user	name, description
site-types	site_type	ref	public	authenticated user	name, description
request-statuses	request_status	ref	public	master role	description
request-types	request_type	ref	public	master role	field, description, comment
users	user	ref	master role	master role	name, dn, email
local-users	atlas_local_user	id	master role	master role	username, first_name, last_name, email, role

6. Special endpoints

GET /

Returns API name, version, documentation URL and resource list.

GET /health

API health only; it is not the full application readiness probe. Response:

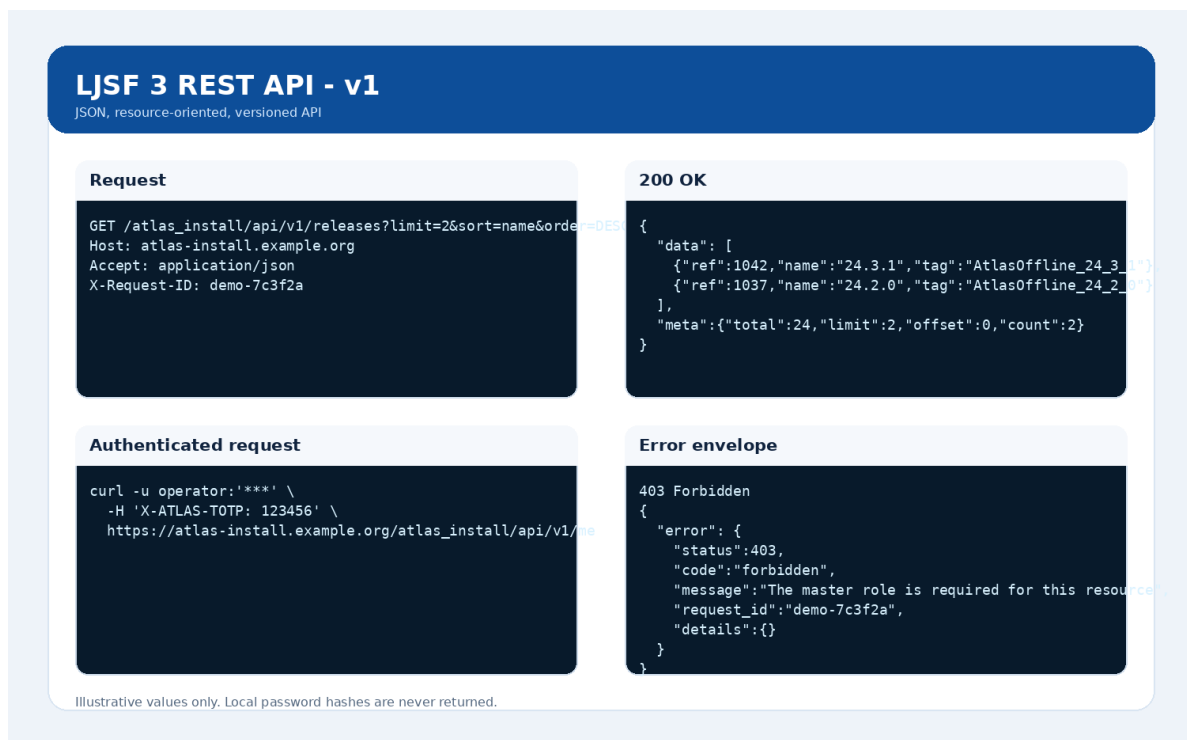
```
{"status": "ok", "api": "v1", "request_id": "..."}.
```

GET /me

Requires authentication and returns the effective identity. `password_hash` is always removed.

GET /openapi

Returns a compact OpenAPI 3.1 document with paths and methods; this manual is the more detailed source for fields, authentication and semantics.



API request/response screenshot

7. Complete resource reference

7.1 releases

DB table: `release_data`

Key: `ref` (integer)

Read access: public

Write access: authenticated user

q search fields: `name`, `tag`, `package`, `comments`

Methods:

- GET `/releases`
- GET `/releases/{id}`
- POST `/releases`
- PATCH `/releases/{id}`
- PUT `/releases/{id}`
- DELETE `/releases/{id}`

Exposed fields:

Field	DB type	Nullable	Writable	Description
<code>ref</code>	<code>int(11)</code>	NO	no	Numeric resource identifier.
<code>name</code>	<code>varchar(50)</code>	NO	yes	Canonical or displayed name.

Field	DB type	Nullable	Writable	Description
build	varchar(15)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
typefk	int(11)	NO	yes	Type reference.
archfk	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
sw_archfk	int(11)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
userfk	int(11)	NO	yes	User reference.
obsolete	int(11)	YES	yes	Obsolete-release flag.
autoinstall	int(11)	YES	yes	Automatic-installation flag.
requires	varchar(30)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
dbrelease	varchar(10)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
installer_version	varchar(10)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
install_tools_version	varchar(10)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
sw_name	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.

Field	DB type	Nullable	Writable	Description
sw_revision	varchar(20)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
sw_physicalpath	varchar(255)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
sw_logicalpath	varchar(255)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
tag	varchar(100)	NO	yes	Software/release tag.
package	varchar(255)	NO	yes	Software package or descriptor.
comments	varchar(255)	YES	yes	Operational comments.
date	datetime	NO	yes	Date/time associated with the record.
cvmfs_available	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
critical	int(11)	NO	yes	Critical-release flag.
critical_validity_from	datetime	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
critical_validity_to	datetime	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
critical_description	varchar(25)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/releases?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/releases/123
```

7.2 sites

DB table: site

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: cs, cename, name, atlas_name, alias, arch

Methods:

- GET /sites
- GET /sites/{id}
- POST /sites
- PATCH /sites/{id}
- PUT /sites/{id}
- DELETE /sites/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
cs	varchar(128)	NO	yes	Legacy computer-service identifier.
cename	varchar(50)	NO	yes	Legacy CE name.
name	varchar(30)	NO	yes	Canonical or displayed name.
atlas_name	varchar(30)	NO	yes	ATLAS site name.
tier_level	int(11)	NO	yes	Site tier level.
gridfk	int(11)	NO	yes	Grid reference.
facilityfk	int(11)	YES	yes	Facility reference.
activity_typefk	int(11)	YES	yes	Activity-type reference.
osname	varchar(128)	NO	yes	OS name.
osrelease	varchar(128)	NO	yes	OS release.
osversion	varchar(128)	NO	yes	OS version.
arch	varchar(20)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.

Field	DB type	Nullable	Writable	Description
tags	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
alias	varchar(128)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
swarea	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
fstype	varchar(15)	YES	yes	Filesystem type.
mountpoint	varchar(255)	YES	yes	Mount point.
capacity	int(11)	YES	yes	Capacity.
available	int(11)	YES	yes	Available space.
quota	int(11)	YES	yes	Quota.
status	int(11)	YES	yes	Application status.
attr	int(10)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
host_status	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
resource_status	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
last_activity	timestamp	YES	yes	Last recorded activity.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/sites?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/sites/123
```

7.3 requests

DB table: request

Key: id (string)

Read access: authenticated user

Write access: authenticated user

q search fields: id, user_comments, admin_comments

Methods:

- GET /requests
- GET /requests/{id}
- POST /requests
- PATCH /requests/{id}
- PUT /requests/{id}
- DELETE /requests/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
id	varchar(128)	NO	no	Resource identifier.
bdiifk	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
sitefk	int(11)	NO	yes	Site reference.
relfk	int(11)	NO	yes	Release reference.
typefk	int(11)	NO	yes	Type reference.
userfk	int(11)	NO	yes	User reference.
adminfk	int(11)	YES	yes	Assigned administrator/operator reference.
statusfk	int(11)	NO	yes	Request-status reference.
force_run	int(11)	NO	yes	Force-execution flag.
request_date	datetime	YES	yes	Request date/time.
update_date	datetime	YES	yes	Last update date/time.
user_comments	varchar(255)	YES	yes	User comments.
admin_comments	varchar(255)	YES	yes	Administrative comments.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/requests?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/requests/REQ-EXAMPLE-001
```

7.4 tasks

DB table: task

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: name, description, target

Methods:

- GET /tasks
- GET /tasks/{id}
- POST /tasks
- PATCH /tasks/{id}
- PUT /tasks/{id}
- DELETE /tasks/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
name	varchar(50)	NO	yes	Canonical or displayed name.
description	varchar(255)	YES	yes	Human-readable description.
target	varchar(20)	YES	yes	Task target.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/tasks?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/tasks/123
```

7.5 architectures

DB table: release_arch

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: platform_type, os_type, gcc_ver, mode, description

Methods:

- GET /architectures
- GET /architectures/{id}

- `POST /architectures`
- `PATCH /architectures/{id}`
- `PUT /architectures/{id}`
- `DELETE /architectures/{id}`

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
platform_type	varchar(16)	NO	yes	Platform type.
os_type	varchar(128)	NO	yes	Operating-system type.
gcc_ver	varchar(128)	NO	yes	GCC/toolchain version.
mode	varchar(10)	NO	yes	Build mode.
description	varchar(255)	NO	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/architectures?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/architectures/123
```

7.6 targets

DB table: `autoinstall_target`

Key: `ref` (integer)

Read access: public

Write access: authenticated user

q search fields: `name`, `description`

Methods:

- `GET /targets`
- `GET /targets/{id}`
- `POST /targets`
- `PATCH /targets/{id}`
- `PUT /targets/{id}`
- `DELETE /targets/{id}`

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.

Field	DB type	Nullable	Writable	Description
name	varchar(15)	NO	yes	Canonical or displayed name.
description	varchar(255)	YES	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/targets?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/targets/123
```

7.7 infosys

DB table: bdii

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: hostname, ns, lb, ld, wmpoxy, myproxy

Methods:

- GET /infosys
- GET /infosys/{id}
- POST /infosys
- PATCH /infosys/{id}
- PUT /infosys/{id}
- DELETE /infosys/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
facilityfk	int(11)	NO	yes	Facility reference.
hostname	varchar(128)	NO	yes	Host name.
port	int(11)	NO	yes	TCP port.
ns	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
lb	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.

Field	DB type	Nullable	Writable	Description
Id	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
wmproxy	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
myproxy	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
preferred	int(11)	NO	yes	Preference flag.
enabled	int(11)	NO	yes	Enable flag (0/1).

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/infosys?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/infosys/123
```

7.8 release-subscriptions

DB table: release_subscription

Key: ref (integer)

Read access: authenticated user

Write access: authenticated user

q search fields: sitename, pattern, comment

Methods:

- GET /release-subscriptions
- GET /release-subscriptions/{id}
- POST /release-subscriptions
- PATCH /release-subscriptions/{id}
- PUT /release-subscriptions/{id}
- DELETE /release-subscriptions/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
sitename	varchar(50)	NO	yes	Site name.

Field	DB type	Nullable	Writable	Description
userfk	int(11)	NO	yes	User reference.
pattern	varchar(50)	NO	yes	Subscription pattern.
date	datetime	NO	yes	Date/time associated with the record.
comment	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/release-subscriptions?
    limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/release-subscriptions/123
```

7.9 release-status

DB table: release_stat

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: name, tag, status, comments

Methods:

- GET /release-status
- GET /release-status/{id}
- POST /release-status
- PATCH /release-status/{id}
- PUT /release-status/{id}
- DELETE /release-status/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
sitefk	int(11)	NO	yes	Site reference.
name	varchar(50)	NO	yes	Canonical or displayed name.
userfk	int(11)	NO	yes	User reference.
pin	int(11)	YES	yes	Pin flag.
pinuserfk	int(11)	YES	yes	User who set the pin.

Field	DB type	Nullable	Writable	Description
pindate	datetime	YES	yes	Pin date.
tag	varchar(100)	NO	yes	Software/release tag.
status	varchar(100)	NO	yes	Application status.
comments	varchar(255)	YES	yes	Operational comments.
date	datetime	NO	yes	Date/time associated with the record.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/release-status?
    limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/release-status/123
```

7.10 grids

DB table: grid

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: name, description

Methods:

- GET /grids
- GET /grids/{id}
- POST /grids
- PATCH /grids/{id}
- PUT /grids/{id}
- DELETE /grids/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
name	varchar(10)	NO	yes	Canonical or displayed name.
description	varchar(255)	YES	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/grids?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/grids/123
```

7.11 facilities

DB table: facility

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: name, description

Methods:

- GET /facilities
- GET /facilities/{id}
- POST /facilities
- PATCH /facilities/{id}
- PUT /facilities/{id}
- DELETE /facilities/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
name	varchar(10)	NO	yes	Canonical or displayed name.
description	varchar(255)	YES	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/facilities?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/facilities/123
```

7.12 site-types

DB table: site_type

Key: ref (integer)

Read access: public

Write access: authenticated user

q search fields: name, description

Methods:

- GET /site-types
- GET /site-types/{id}
- POST /site-types

- PATCH /site-types/{id}
- PUT /site-types/{id}
- DELETE /site-types/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
name	varchar(50)	NO	yes	Canonical or displayed name.
description	varchar(255)	NO	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/site-types?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/site-types/123
```

7.13 request-statuses

DB table: request_status

Key: ref (integer)

Read access: public

Write access: master role

q search fields: description

Methods:

- GET /request-statuses
- GET /request-statuses/{id}
- POST /request-statuses
- PATCH /request-statuses/{id}
- PUT /request-statuses/{id}
- DELETE /request-statuses/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
description	varchar(128)	YES	yes	Human-readable description.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/request-statuses?
    limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/request-statuses/123
```

7.14 request-types

DB table: request_type

Key: ref (integer)

Read access: public

Write access: master role

q search fields: field, description, comment

Methods:

- GET /request-types
- GET /request-types/{id}
- POST /request-types
- PATCH /request-types/{id}
- PUT /request-types/{id}
- DELETE /request-types/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
level	int(11)	NO	yes	Level/precedence.
field	varchar(100)	NO	yes	Application field name.
description	varchar(128)	YES	yes	Human-readable description.
comment	varchar(255)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/request-types?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/request-types/123
```

7.15 users

DB table: user

Key: ref (integer)

Read access: master role

Write access: master role

q search fields: name, dn, email

Methods:

- GET /users
- GET /users/{id}
- POST /users
- PATCH /users/{id}
- PUT /users/{id}
- DELETE /users/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
ref	int(11)	NO	no	Numeric resource identifier.
name	varchar(50)	NO	yes	Canonical or displayed name.
dn	varchar(200)	YES	yes	X.509 Distinguished Name.
email	varchar(150)	YES	yes	Associated email address.
priv_view	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
priv_insert	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
priv_update	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
priv_pin	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
priv_relsb	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.

Field	DB type	Nullable	Writable	Description
priv_critical	int(11)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
rolefk	int(11)	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
valid_start	datetime	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
valid_end	datetime	YES	yes	Canonical application-schema field; operational meaning depends on the resource.
enabled	int(11)	NO	yes	Enable flag (0/1).

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/users?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/users/123
```

7.16 local-users

DB table: atlas_local_user

Key: id (integer)

Read access: master role

Write access: master role

q search fields: username, first_name, last_name, email, role

Methods:

- GET /local-users
- GET /local-users/{id}
- POST /local-users
- PATCH /local-users/{id}
- PUT /local-users/{id}
- DELETE /local-users/{id}

Exposed fields:

Field	DB type	Nullable	Writable	Description
id	BIGINT	NO	no	Resource identifier.

Field	DB type	Nullable	Writable	Description
username	VARCHAR(64)	NO	yes	Local account name.
first_name	VARCHAR(100)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
last_name	VARCHAR(100)	NO	yes	Canonical application-schema field; operational meaning depends on the resource.
email	VARCHAR(254)	NO	yes	Associated email address.
role	VARCHAR(32)	NO	yes	Local application role: user, admin, or master.
enabled	TINYINT(1)	NO	yes	Enable flag (0/1).
must_change_password	TINYINT(1)	NO	yes	When 1, password change is required before API use.
created_at	DATETIME	NO	yes	Creation date/time.
updated_at	DATETIME	NO	yes	Last update date/time.

Examples:

```
curl https://atlas-install.example.org/atlas_install/api/v1/local-users?limit=20&offset=0
curl https://atlas-install.example.org/atlas_install/api/v1/local-users/123
```

`password` is a special input-only field: accepted by POST/PATCH/PUT, minimum 8 characters, converted into `password_hash`, and never returned.

8. End-to-end examples

8.1 Create and update a local user

```
curl -u admin:'ADMIN_PASSWORD' -H 'Content-Type: application/json' \
-d '{"username":"operator","first_name":"API","last_name":"Operator","email":"operator@example.org","role":"user","enabled":1,"password":"StrongPass-2026"}' \
https://atlas-install.example.org/atlas_install/api/v1/local-users

curl -u admin:'ADMIN_PASSWORD' -X PATCH -H 'Content-Type: application/json' \
-d '{"role":"admin","must_change_password":0}' \
https://atlas-install.example.org/atlas_install/api/v1/local-users/5
```

8.2 Python

```
import requests

base = 'https://atlas-install.example.org/atlas_install/api/v1'
r = requests.get(f'{base}/releases', params={'q':'24.3','limit':50}, timeout=30)
r.raise_for_status()
for release in r.json()['data']:
    print(release['ref'], release['name'], release['tag'])
```

8.3 Robust error handling

```
import requests

r = requests.get('https://atlas-install.example.org/atlas_install/api/v1/requests',
                 timeout=30)
if r.status_code >= 400:
    payload = r.json()
    err = payload.get('error', {})
    print('HTTP', r.status_code, err.get('code'), err.get('message'), 'request_id=',
          err.get('request_id'))
```

9. Security, compatibility and current limitations

- Always use HTTPS; Basic sends reusable credentials on every request.
- Local passwords/hashes are never returned.
- Writes use parameterized queries and an exposed-field allowlist.
- There are currently no ETag/If-Match semantics and no Idempotency-Key header: clients must avoid blind POST retries.
- There are no bulk endpoints: each request modifies one resource.
- PUT is intentionally patch-like in v1; do not assume full replacement semantics.
- Maximum page size is 1000; iterate with offset for larger datasets.

- Resources with DB dependencies may return 409 on DELETE/UPDATE due to constraints/conflicts.
- Legacy endpoints are separate; do not infer REST contracts from legacy endpoint syntax.

10. Production client checklist

- ☐ Verify the server TLS certificate.
- ☐ Set explicit client-side timeouts.
- ☐ Record and propagate `X-Request-ID` where possible.
- ☐ Handle 401, 403, 409, 415 and 5xx separately.
- ☐ Never log passwords, Basic Authorization or TOTP codes.
- ☐ Use pagination and reasonable limits.
- ☐ Before DELETE, account for dependencies and handle 409.
- ☐ Use dedicated automation identities with the least required role.

Appendix C. Glossary

Term	Definition
BDII / InfoSys	Information service used to discover resources/grid endpoints.
CRL	Certificate Revocation List used for client-certificate validation.
IGTF	Trust federation/CA distribution used by the trust store.
RWO	ReadWriteOnce: a volume writable from one node at a time.
TOTP	Time-based One-Time Password.
TLS passthrough	Ingress forwards the TLS stream without termination, preserving the client certificate to Apache.
Legacy endpoint	Historical interface outside the REST v1 contract.
Request ID	Correlation identifier returned/logged for a request.

X.509 identity, CA binding and historical user retention

For a session authenticated with a personal certificate, **Current user details** shows the subject DN, issuing CA, CA DN, whether the user is known to the application, and the effective role. If the DN is known but the presented certificate was issued by a different CA from the one stored for the account, the UI highlights the mismatch and no role is assigned. The same section explains when a role is missing because the account is disabled, expired, pending approval, or has no active role.

Historical records that do not yet contain CA information are marked as **legacy CA-unbound**. For backward compatibility, the role remains recognized until the user explicitly updates the profile while presenting the current certificate, at which point the CA is bound to the record.

A local `master` session is a first-class administrative identity: `protected/user.php` and other protected pages do not require a certificate when the local account has the required role. Legacy compatibility is provided through a `LOCAL:<username>` row in the historical `user` table, while authorization decisions always use the current local session rather than TLS fields.

From `protected/user.php`, a `master` administrator can delete an X.509 user. Deletion is **soft**: the `user` row is retained, disabled, and marked with a deletion timestamp. Foreign keys and historical references in requests, logs, releases, tasks, and other records continue to resolve the original identity.

Local sessions are explicitly identified as **Local user** and are never described as certificate sessions. Authorization checks on protected pages use the unified identity and honor the local `user`, `admin`, or `master` role.